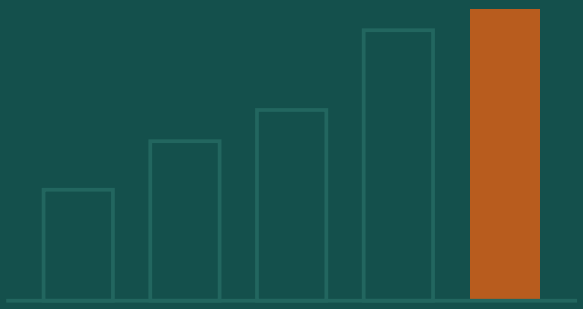
LAB 8 · AN INTRODUCTION

Evals

How you find out whether the change you just made actually made anything better.



FOR	Anyone who has improved a prompt by feel.
READ IT	Before Sunday afternoon. Ten minutes.

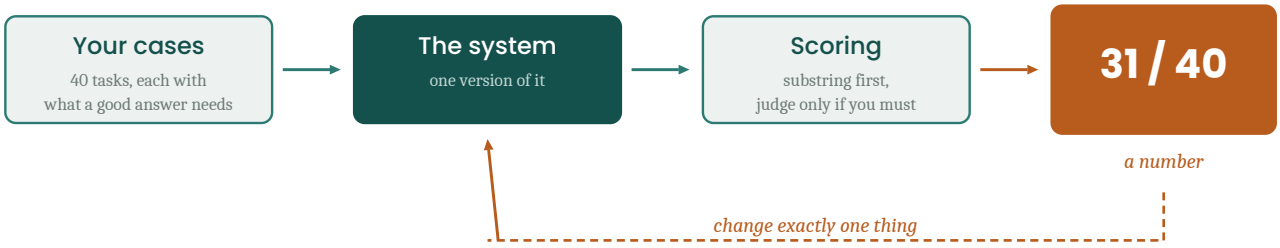
What it actually is

A set of test cases, a way of scoring the answers, and a number. You change one thing, run it again, and watch the number move.

That is the whole mechanism, and the reason it needs explaining is not that it is complicated. It is that almost nobody does it, and the alternative feels fine right up until it doesn't.

The alternative is tuning by vibes. You change a prompt. You try it once. It reads better. You ship it. You have no idea whether you fixed one case and broke two others, because you only looked at one case, and it was the one you were already thinking about.

Change one thing. Re-measure. Keep the number.



Forty cases is a realistic start. The common mistake is assuming you need hundreds, which is why most people never begin.

> **WHY THIS IS THE UNGLAMOROUS HALF OF THE FIELD**
>
> Building something that works once is a demo, and demos are easy now. Knowing that it still works after a change - and being able to prove it to someone else - is the part that separates a prototype from a system, and almost nobody has it.

Example 1 - Making something measurably better

The ordinary case: you have something that half works.

Support tickets arrive and a model routes them to the right team. It is roughly right, and "roughly" is doing a lot of work in that sentence - nobody can say how roughly, because nobody has counted.

So you count. Forty real tickets, each with the team it should have gone to. One afternoon of work, mostly copy and paste. Then:

Baseline	**23 / 40.** Worse than anyone guessed. Three people had described it as "pretty good".

+ team descriptions	**29 / 40.** Telling it what each team actually owns, in one sentence each. The biggest single win, and the cheapest.
+ five examples	**31 / 40.** A small gain for a large number of tokens on every request. Worth knowing before you pay for it forever.
+ an "unclear" option	**36 / 40.** Letting it decline to route. Most of the remaining errors were tickets that genuinely could have gone to two teams.

Four numbers, and each one changed exactly one thing. That last row is the interesting one: the fix was not a better prompt, it was noticing that some tickets are genuinely ambiguous and the system had no way to say so. **You cannot find that by reading outputs. You find it by looking at which cases keep failing.**

> **THE SENTENCE THIS BUYS YOU**
>
> "I took it from 58% to 90%, and here is which change bought each increment." That is a different conversation from "I improved the prompt", in a standup, in a review, and especially in an interview.

Example 2 - The day the model changes

The case people do not see coming, and it is now routine.

Your system has been running happily for eight months. Then one of these happens, and at least one of them will:

A new version ships.	Better on every published benchmark. Also different, in ways no benchmark measured, on your particular task.
Costs come under review.	Someone asks whether the cheaper model would do. A perfectly reasonable question that you currently cannot answer.
A version is retired.	You are moving whether you like it or not, on somebody else's timetable.

Without an eval suite, each of these is a gamble dressed up as a decision. You swap the model, click around for ten minutes, and ship. Any regression is discovered by customers, weeks later, in a form that does not obviously point back at the change.

With one, it is twenty minutes and a table:

YOU	**python eval.py --compare**
OUTPUT	`current 36/40 (90%) 412k tokens $1.84`
OUTPUT	`new 38/40 (95%) 401k tokens $1.79`
OUTPUT	`cheaper 31/40 (78%) 398k tokens $0.31`
AGENT	Regressions on the cheaper model are concentrated in the ambiguous-ticket cases - it routes confidently where the current one declines.

Now the decision is a decision. The new version is a free upgrade. The cheaper one saves 83% of the bill and loses the exact behaviour you worked hardest to get - which might still be the right trade for a low-stakes queue, and you can now make that call deliberately rather than by accident.

How to score without fooling yourself

Most of the craft is here, and the rule is simple: use the cheapest check that can tell the difference.

Reach for these in order

1. **Exact or substring match**
   Free, instant, never drifts, needs no model. Use it wherever the answer contains a checkable fact.

2. **A rule you write**
   A few lines of Python: is it valid JSON, is the total right, is the citation real. Still free, still deterministic.

3. **A model as judge**
   Necessary for prose. Also biased toward long answers, inconsistent between runs, and a cost per case.

IF YOU USE A JUDGE, CHECK THE JUDGE

Grade ten cases yourself and compare. A judge you have never audited is not a measurement - it is a second opinion from something with the same blind spots as the thing it is grading.

The case most suites are missing

Every suite tests things the system should get right. Very few test what happens when the answer **is not available** - the question whose data does not exist, the ticket with a false premise, the field that is genuinely absent from the document.

A system that answers 95% of questions well and invents the other 5% without flagging them is worse than one that answers 80% and says so, because with the first one you cannot tell which answers to trust. Put three unanswerable cases in your suite from the start.

What to take into the lab

ONE

Cases, a scoring function, and a number. Forty cases is a real eval suite; the belief that you need hundreds is why most people have none.

TWO

Change one thing at a time, or the number tells you nothing about which change did what.

THREE

Test what happens when the answer is not available. That is the failure that destroys trust, and it never appears in happy-path testing.

In the lab you will build a twelve-case suite against the agent from Saturday, run three variants of its prompt, and watch the number move. Then you will add three cases of your own and find out whether you can predict which ones it fails - being wrong about that is the most useful thing that can happen in the hour.